

Deep Learning for Solving Dynamic Models

Session 8: Economics-informed neural networks, foundations

Simon Scheidegger

HEC, University of Lausanne

Based on Raissi, Perdikaris & Karniadakis (2019), *Journal of Computational Physics* 378, 686–707

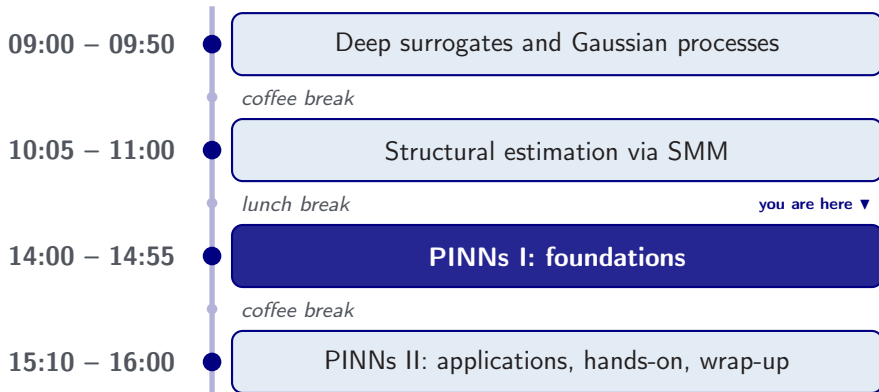
University of Padova

September 24, 2026

Further: Sirignano & Spiliopoulos (2018), Deep Galerkin Method for PDEs in high dimensions.

Al-Aradi, Correia, Naez, Hoshisashi & Schwab (2018), DGM PDE solver.

Day 2 at a Glance



Outline of the PINN Block, Sessions 8 and 9

1. **From DEQNs to PINNs**

Bridging discrete and continuous time; the PINN loss.

2. **PINN Fundamentals**

Automatic differentiation for PDEs, collocation, training loop.

3. **Soft vs. Hard Boundary Conditions**

Trial solutions, 2D Poisson, practical guidance.

4. **Economic Applications: HJB Equation**

Cake-eating problem, scaled trial solution + hard BCs, Adam $\rightarrow$ L-BFGS, analytical benchmark; DGM as optional high-D architecture.

5. **Financial Applications: Black–Scholes**

Option pricing via PINNs, comparison with analytical formula.

6. **When to Use One, and What Goes Wrong**

PINNs vs. grid solvers, inverse problems, failure modes, and which notebooks to run.

Roadmap of This Session (55 min)

Part I of two. This session builds the method; Session 9 after the break applies it to economic and financial models.

I. From DEQNs to PINNs (12 min)

- ▶ Discrete vs. continuous time, the PINN loss
- ▶ Finger exercise

II. PINN fundamentals (20 min)

- ▶ Autograd for PDEs, collocation, the training loop
- ▶ Adam $\rightarrow$ L-BFGS in FP64; warm-up 1D ODE

III. Boundary conditions (23 min)

- ▶ Soft penalties vs. hard trial solutions
- ▶ Hard BCs in 2D; the Poisson equation; finger exercise

Part I

From DEQNs to PINNs

Recall: The DEQN Loss from Day 1

Yesterday morning (02_DeepEquilibriumNets.pdf) **we trained** a neural network $\mathcal{N}_\theta$ to satisfy economic equilibrium conditions in **discrete time**:

$$\ell_\theta^{\text{DEQN}} = \frac{1}{N} \sum_{i=1}^N \|G(\mathbf{x}_i, \mathcal{N}_\theta(\mathbf{x}_i))\|^2$$

- ▶ $G(\cdot) = 0$: Euler equations, budget constraints, market clearing
- ▶ Training points $\mathbf{x}_i$: simulated from the model's own dynamics
- ▶ **No labeled data needed**: structural equations *are* the loss

Today's question: What if the economic model is formulated in **continuous time**?

Discrete vs. Continuous Time

Discrete Time (DEQN)

Euler equation:

$$u'(c_t) = \beta \mathbb{E}[u'(c_{t+1}) R_{t+1}]$$

State transition:

$$\mathbf{x}_{t+1} = h(\mathbf{x}_t, p(\mathbf{x}_t), \varepsilon_{t+1})$$

Loss: algebraic residual $G(\cdot) = 0$

Continuous Time (PINN)

State evolution:

$$da = (ra - c) dt + \sigma dW$$

HJB equation:

$$\rho V = \max_c \{ u(c) + V_a(ra - c) + \frac{1}{2}\sigma^2 V_{aa} \}$$

Loss: PDE residual $\mathcal{R}(\cdot) = 0$

Key shift: from algebraic residuals to **differential** residuals, we need **derivatives** of the neural network. The diffusion term $\frac{1}{2}\sigma^2 V_{aa}$ is why **second** derivatives matter.

Why Continuous Time Matters

Many frontier models are formulated in continuous time:

- ▶ **Heterogeneous-agent models:** Achdou, Han, Lasry, Lions & Moll (2022), income and wealth distribution dynamics via HJB–KFE systems
- ▶ **Mean-field games:** optimal policy with a continuum of interacting agents
- ▶ **Asset pricing:** Black–Scholes, Heston, jump-diffusion models
- ▶ **Macro-finance:** Brunnermeier & Sannikov (2014), He & Krishnamurthy (2013)

Where this sits in the course

Day 1 solved equilibrium conditions in **discrete** time with DEQNs. This afternoon we do the **continuous-time** counterpart: this session builds the method on equations with known solutions, Session 9 applies it to the cake-eating HJB, to Black–Scholes, and to a consumption-savings problem with a borrowing constraint and a discrete income state, solved by finite differences and by a PINN side by side.

The PINN Loss Function

Core idea (Raissi, Perdikaris & Karniadakis, 2019):

Given a PDE $\mathcal{D}[u](\mathbf{x}) = 0$ on domain Ω with boundary conditions $\mathcal{B}[u](\mathbf{x}) = 0$ on $\partial\Omega$, define:

$$\ell_{\theta}^{\text{PINN}} = \underbrace{\frac{1}{N_r} \sum_{i=1}^{N_r} |\mathcal{D}[\mathcal{N}_{\theta}](\mathbf{x}_i^r)|^2}_{\text{PDE residual}} + \underbrace{\frac{\lambda}{N_b} \sum_{j=1}^{N_b} |\mathcal{B}[\mathcal{N}_{\theta}](\mathbf{x}_j^b)|^2}_{\text{boundary/initial conditions}}$$

- ▶ $\mathcal{N}_{\theta}$: neural network with parameters θ
- ▶ $\mathbf{x}_i^r$: **collocation points** in the interior of Ω
- ▶ $\mathbf{x}_j^b$: points on the boundary $\partial\Omega$
- ▶ Derivatives $\mathcal{D}[\mathcal{N}_{\theta}]$ computed via **automatic differentiation**

Warm-up ODE: for $u''(x) = -1$ on $(0, 1)$ with $u(0) = u(1) = 0$, use $\mathcal{D}[u] = u'' + 1$ in the interior and $\mathcal{B}[u] = (u(0), u(1))$ on the boundary.

Finger Exercise: Writing a PINN Loss

Exercise (5 min)

Consider the ODE $u''(x) = -1$ on $[0, 1]$ with boundary conditions $u(0) = u(1) = 0$.

1. Write down the PINN residual $\mathcal{R}(x; \theta) = \dots$ using a neural network $\mathcal{N}(x; \theta)$.
2. Write the total PINN loss as a sum of a PDE residual term and a boundary term.
3. What is the exact solution?

Solution: Writing a PINN Loss

Answer

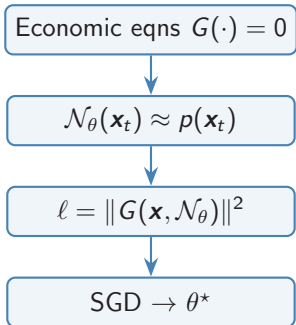
1. $\mathcal{R}(x; \theta) = \frac{\partial^2 \mathcal{N}}{\partial x^2}(x; \theta) + 1.$

2. $\mathcal{L}(\theta) = \underbrace{\frac{1}{N_r} \sum_{i=1}^{N_r} |\mathcal{R}(x_i; \theta)|^2}_{\text{PDE residual}} + \lambda \underbrace{[\mathcal{N}(0; \theta)^2 + \mathcal{N}(1; \theta)^2]}_{\text{boundary conditions}}$

3. $u(x) = \frac{1}{2}x(1 - x)$ satisfies $u'' = -1$, $u(0) = u(1) = 0$. ✓

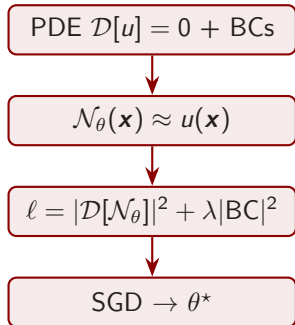
DEQN vs. PINN: Side-by-Side

DEQN (Discrete Time)



← same principle →
+ autograd derivatives

PINN (Continuous Time)



Both approaches are **unsupervised**: the physics (economics) is the loss function. The key addition for PINNs: we differentiate the network via any auto-diff tool (e.g. `torch.autograd.grad`).

Part II

PINN Fundamentals

Automatic Differentiation for PDEs

Key tool: `torch.autograd.grad` with `create_graph=True`.

```
# First derivative dy/dx
dydx = torch.autograd.grad(
    y, x,
    grad_outputs=torch.ones_like(y),
    create_graph=True # keep graph for higher derivs
)[0]
```

- ▶ `create_graph=True`: retains the computational graph so we can differentiate **again**
- ▶ This gives us **algorithmic** derivatives (up to floating-point precision), no finite differences!
- ▶ Works for **any order**: $\frac{\partial^n}{\partial x^n}$ by chaining n calls

Computing Second Derivatives

Chain two autograd calls to get $y''(x)$:

First derivative

```
dydx = torch.autograd.grad(  
    y, x, grad_outputs=torch.ones_like(y),  
    create_graph=True)[0]
```

Second derivative

```
d2ydx2 = torch.autograd.grad(  
    dydx, x, grad_outputs=torch.ones_like(dydx),  
    create_graph=True)[0]
```



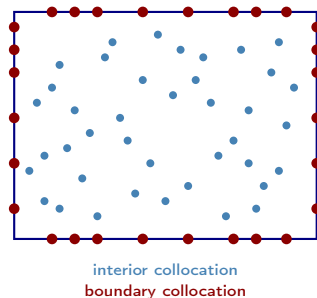
Collocation Points

PINNs evaluate the PDE residual at **randomly sampled** points, no grid required.

1D: interior + boundary



2D: $\Omega = (0, 1)^2$



Points are **resampled** periodically during training to avoid overfitting to a fixed set.

PINN Training Algorithm

Algorithm: Physics-Informed Neural Network Training

Input: PDE $\mathcal{D}[u] = 0$, BCs $\mathcal{B}[u] = 0$, network $\mathcal{N}_\theta$, learning rate η

for epoch = $1, \dots, E$ **do**

 Sample N_r collocation points $\{\mathbf{x}_i^r\}$ in domain Ω

 Sample N_b boundary points $\{\mathbf{x}_j^b\}$ on $\partial\Omega$

 Forward pass: compute $\mathcal{N}_\theta(\mathbf{x}_i^r)$ and $\mathcal{N}_\theta(\mathbf{x}_j^b)$

 Autograd: compute $\mathcal{D}[\mathcal{N}_\theta](\mathbf{x}_i^r)$ via `torch.autograd.grad`

$\ell_\theta = \frac{1}{N_r} \sum_i |\mathcal{D}[\mathcal{N}_\theta](\mathbf{x}_i^r)|^2 + \frac{\lambda}{N_b} \sum_j |\mathcal{B}[\mathcal{N}_\theta](\mathbf{x}_j^b)|^2$

 Update: $\theta \leftarrow \theta - \eta \nabla_\theta \ell_\theta$

end for

Output: Trained $\mathcal{N}_{\theta^*}$ approximating the PDE solution

Note: Unlike DEQNs, training points are **not** simulated from model dynamics, they are drawn uniformly (or adaptively) from the domain.

Adam First, Then L-BFGS, in Double Precision

Two stages.

- ▶ **Adam** on resampled points finds the basin; every step sees a different loss, so it cannot go below a noise floor.
- ▶ **L-BFGS** on a *fixed* set of points: deterministic objective, strong-Wolfe line search, the residual closes. Resampling inside it breaks the line search.

Measured, hard-BC run of 08_02, max error: 1.0×10^{-5} after 5,000 Adam steps, 2.6×10^{-7} after L-BFGS on 200 fixed points.

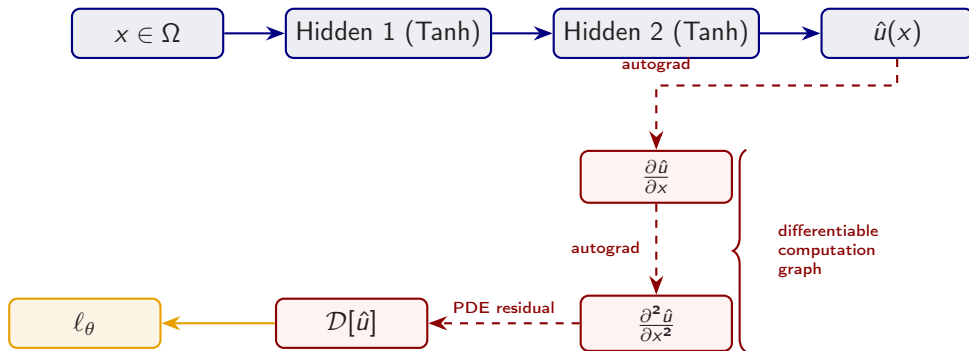
Practical recipe

1. float64 for the model and every tensor.
2. Adam, lr 10^{-3} , a few thousand steps, resampled points.
3. Freeze the points; `torch.optim.LBFGS, strong_wolfe`.
4. Stop when the residual stagnates.

Why double precision

Same run in float32: L-BFGS stops after 55 evaluations at a loss of 3×10^{-9} , the resolution of the objective, max error 6.8×10^{-7} ; in float64 it reaches 8×10^{-10} and 2.6×10^{-7} . On a CPU it costs nothing. Used from 08_02 on; precision analysis in [arXiv:2505.10949](#).

PINN Architecture



The entire pipeline, forward pass *and* derivative computation, is **differentiable**, enabling end-to-end gradient-based training.

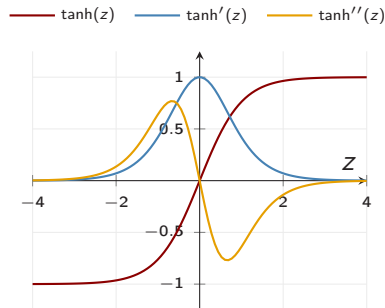
Why Tanh Activations for PINNs?

PDE residuals require smooth derivatives.

- ▶ **ReLU**: C^0 only; its second derivative vanishes almost everywhere, so a u'' residual carries no gradient
- ▶ **Tanh**: C^∞ , all derivatives are well-defined and informative
- ▶ **Softplus**: C^∞ , useful for positivity constraints (e.g. consumption)

Rule of thumb

For PINNs requiring k -th order derivatives, use activations that are at least C^k .



Warm-up: Solving an ODE with a PINN

Problem: $y''(x) = -1$ on $(0, 1)$, $y(0) = y(1) = 0$. **Analytical:** $y(x) = \frac{1}{2}x(1 - x)$.

PINN setup:

1. NN $\mathcal{N}_\theta : [0, 1] \rightarrow \mathbb{R}$, 2 hidden layers of 20 units, tanh
2. Residual: $r(x) = \mathcal{N}_\theta''(x) + 1$
3. Loss: PDE residual + BC penalty (soft, here)

$$\ell = \frac{1}{N} \sum_i r(x_i)^2 + \mathcal{N}_\theta(0)^2 + \mathcal{N}_\theta(1)^2$$

After 5,000 Adam steps (08_02's recipe comes next):

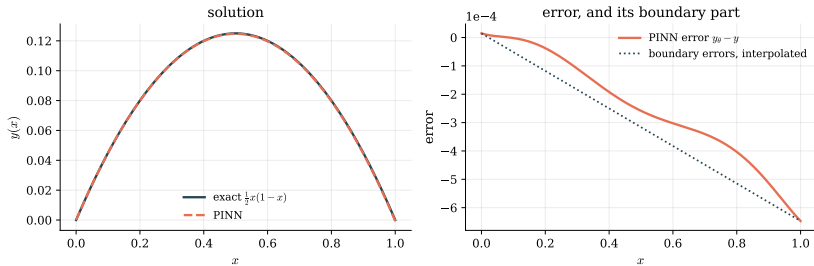
- Loss 8×10^{-6} , of which the boundary term is 3×10^{-7}
- Max absolute error 6.5×10^{-4} , mean 2.5×10^{-4}

Read the error, not the loss

A small loss says the residual is small at the sampled points. Only the comparison against $\frac{1}{2}x(1 - x)$ says the *solution* is right. Here the loss is 10^{-5} and the error 10^{-3} .

Notebook: 08_01_ODE_PINN_ZeroBCs

Warm-up: the Error Is the Boundary Error, Carried Inside



Left: the network on top of $y(x) = \frac{1}{2}x(1-x)$. Right: the error $y_\theta - y$ (solid) and the straight line through its two boundary values (dotted). With the residual at 10^{-5} , $e'' \approx 0$: the miss of 6.5×10^{-4} at $x = 1$ is carried through the domain, and the error deviates from the straight line by at most 1.2×10^{-4} . A soft penalty puts the boundary error everywhere. Part III removes it.

Notebook: 08_01_ODE_PINN_ZeroBCs

Non-Zero Boundary Conditions: The Challenge

Problem: $y''(x) = -1$ on $(0, 1)$ with $y(0) = 1$, $y(1) = 2$.

Analytical solution: $y(x) = -\frac{1}{2}x^2 + \frac{3}{2}x + 1$.

Soft BC enforcement:

$$\ell = \frac{1}{N} \sum_i r(x_i)^2 + \lambda_0 (\mathcal{N}_\theta(0) - 1)^2 + \lambda_1 (\mathcal{N}_\theta(1) - 2)^2$$

Issues with soft enforcement:

- ▶ BCs are only **approximately** satisfied
- ▶ Penalty weight λ requires **tuning**: too small $\Rightarrow$ BC violated; too large $\Rightarrow$ PDE ignored
- ▶ Boundary errors can **propagate** into the interior

$\Rightarrow$ **Motivates hard boundary enforcement** (Part III).

Part III

Soft vs. Hard Boundary Conditions

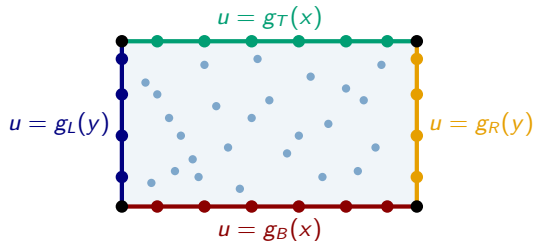
What Are Boundary Conditions?

1D: $y''(x) = -1$ on $(0, 1)$



Interior: enforce $y'' + 1 = 0$
Boundary: enforce $y = \text{prescribed value}$

2D: $\Delta u = f$ on $(0, 1)^2$

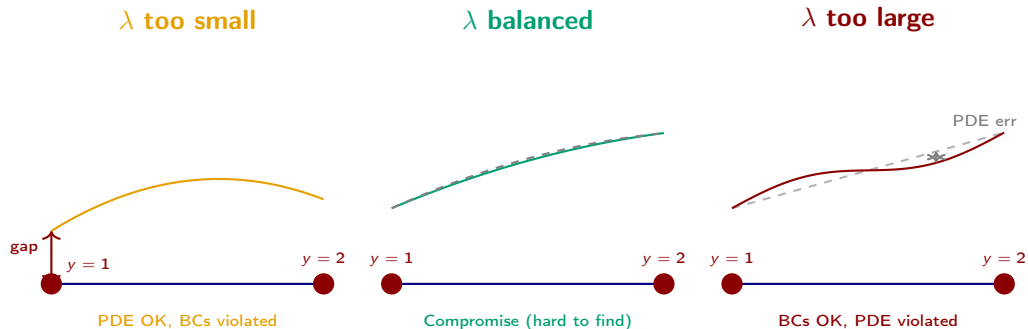


Two enforcement strategies

Soft: add BC violation as a penalty term in the loss.

Hard: build BCs into the network output by construction.

The Problem with Soft Boundary Conditions



Soft BCs require **careful balancing** of the penalty weight λ .
Too small: BCs violated. Too large: PDE ignored. Just right: hard to find *a priori*.

Hard BC Enforcement: Trial Solutions

Idea: construct a **trial solution** that satisfies BCs *by construction*:

$$\hat{y}(x) = A(x) + B(x) \cdot \mathcal{N}_\theta(x)$$

where:

- ▶ $A(x)$: **anchor function** satisfying the BCs exactly, e.g. $A(x) = 1 + x$
- ▶ $B(x)$: **mask function** vanishing at boundaries, e.g. $B(x) = x(1 - x)$
- ▶ $\mathcal{N}_\theta(x)$: free neural network (no BC constraints)

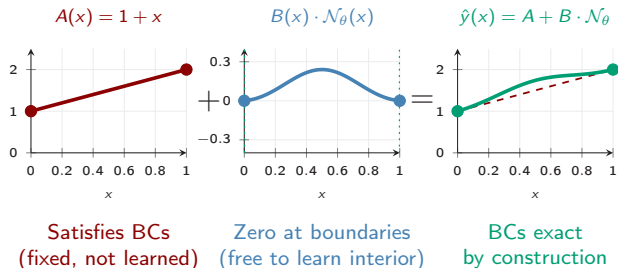
Verification:

$$\hat{y}(0) = A(0) + B(0) \cdot \mathcal{N}_\theta(0) = 1 + 0 = 1 \quad \checkmark$$

$$\hat{y}(1) = A(1) + B(1) \cdot \mathcal{N}_\theta(1) = 2 + 0 = 2 \quad \checkmark$$

$\Rightarrow$ BCs hold **identically**, only the PDE residual loss remains.

Trial Solution: Visual Decomposition



Hard BC: Implementation

```
def A(x): # Anchor: A(0)=1, A(1)=2
    return 1.0 + x
def B(x): # Mask: B(0)=B(1)=0
    return x * (1.0 - x)

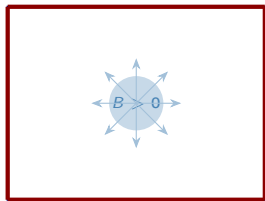
def y_trial(model, x): # Hard-BC trial solution
    return A(x) + B(x) * model(x)

def residual(model, x):
    x.requires_grad_(True)
    y = y_trial(model, x)
    dydx = torch.autograd.grad(y, x, ...)[0]
    d2ydx2 = torch.autograd.grad(dydx, x, ...)[0]
    return d2ydx2 + 1.0 # should be 0
```

Advantage: the loss has **only one term** (PDE residual), no λ to tune!

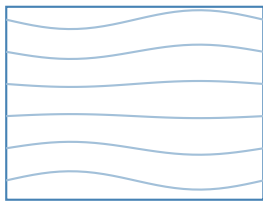
Hard BCs in 2D: Building the Trial Solution

Mask $B(x, y) = x(1-x)y(1-y)$



$B = 0$ on all edges

Network $\mathcal{N}_\theta(x, y)$



Free to take any value



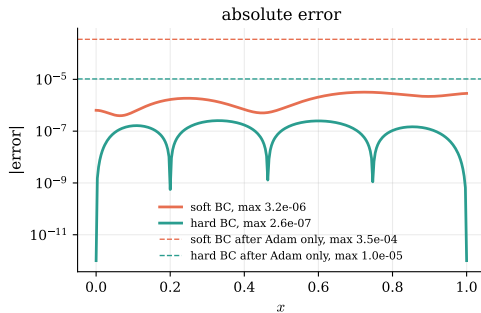
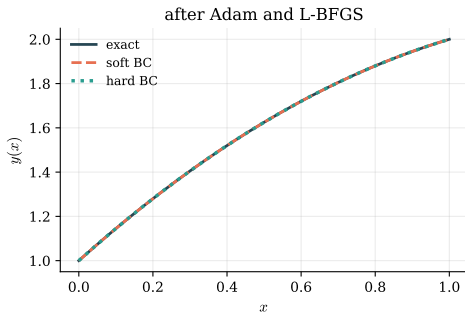
$B \cdot \mathcal{N}_\theta$: zero on $\partial\Omega$



Exactly zero on boundary

Then $\hat{u}(x, y) = \underbrace{A(x, y)}_{\text{matches BCs}} + \underbrace{B(x, y) \cdot \mathcal{N}_\theta(x, y)}_{\text{zero on } \partial\Omega}$ satisfies Dirichlet BCs **exactly**.

Soft vs. Hard BCs: the Same Run, Twice



Same network, seed, points and steps; only the boundary treatment differs. Max error after 5,000 Adam steps: soft 3.5×10^{-4} , hard 1.0×10^{-5} (dashed lines). After the L-BFGS stage: soft 3.2×10^{-6} , hard 2.6×10^{-7} (solid curves). The hard run is exact at the endpoints by construction; the soft run's error is largest there.

Notebook: 08_02_ODE_PINN_SoftVsHardBCs

Finger Exercise: Hard Boundary Conditions

Exercise (5 min)

Consider the same ODE $u''(x) = -1$ on $[0, 1]$ with $u(0) = u(1) = 0$.

1. Construct a **hard-BC trial solution** of the form $\hat{u}(x) = B(x) \cdot \mathcal{N}(x; \theta)$, where $B(x)$ enforces both BCs exactly. (Hint: B must vanish at $x = 0$ and $x = 1$.)
2. Verify that $\hat{u}(0) = \hat{u}(1) = 0$ for *any* network output $\mathcal{N}$.
3. What is the advantage of hard BCs over soft BCs?

Solution: Hard Boundary Conditions

Answer

1. $\hat{u}(x) = x(1 - x) \cdot \mathcal{N}(x; \theta)$. Since $B(x) = x(1 - x)$ vanishes at both endpoints, the BCs are satisfied exactly.
2. $\hat{u}(0) = 0 \cdot \mathcal{N}(0; \theta) = 0$. $\hat{u}(1) = 0 \cdot \mathcal{N}(1; \theta) = 0$. ✓
This holds regardless of what $\mathcal{N}$ outputs.
3. Hard BCs **eliminate the boundary loss term** entirely, the optimizer only needs to minimize the PDE residual. This reduces the number of hyperparameters (λ is gone) and often speeds up convergence.

Extension to 2D: Poisson Equation

Problem: $\Delta u = f(x, y)$ on $\Omega = (0, 1)^2$ with Dirichlet BCs.

Manufactured solution: $u^*(x, y) = x^2 + y + \sin(\pi x) \sin(\pi y)$.

Hard BCs via transfinite (Coons-patch) interpolation: the standard 4-side construction that matches each boundary function exactly along its edge:

$$\hat{u}(x, y) = A(x, y) + \underbrace{x(1-x)y(1-y)}_{B(x,y)} \cdot \mathcal{N}_\theta(x, y)$$

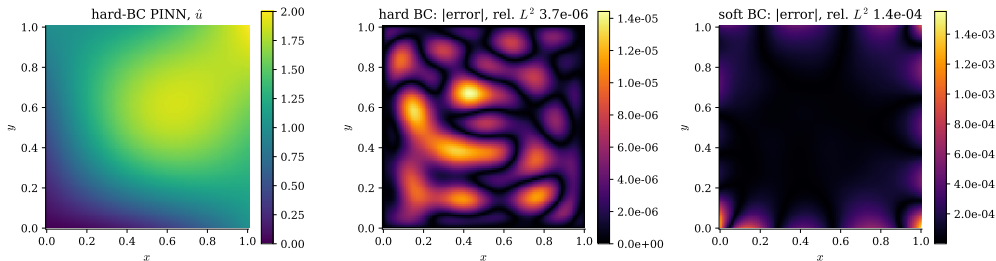
where $A(x, y)$ interpolates boundary data from all four sides:

$$A = (1-x)g_L(y) + xg_R(y) + (1-y)g_B(x) + yg_T(x) - \text{corners}$$

Laplacian via autograd:

$$\Delta \hat{u} = \frac{\partial^2 \hat{u}}{\partial x^2} + \frac{\partial^2 \hat{u}}{\partial y^2} \quad (\text{two chains of autograd.grad})$$

2D Poisson: Results



Left: the hard-BC solution. Middle and right: the absolute error of the hard and of the soft run, same network, seed, sampling and schedule (5,000 Adam steps, then L-BFGS on a fixed set). Relative L^2 error 3.7×10^{-6} against 1.4×10^{-4} : a factor **38**, at one fewer hyperparameter. The soft run's largest error, 1.6×10^{-3} , sits on the boundary; the hard run's boundary error is 10^{-16} .

Notebook: 08_03_PDE_PINN_Poisson2D

Soft vs. Hard BCs: When to Use Which?

	Soft BCs	Hard BCs
Best for	Irregular domains, Robin or Neumann BCs	Simple domains, Dirichlet BCs, high accuracy
Pros	Flexible, easy to implement	Exact BCs, fewer hyperparams, better accuracy
Cons	λ tuning, approximate BCs	Needs analytical A , B ; harder for irregular domains

Practical recommendation

Start with **hard BCs** when the domain and boundary data allow it. Fall back to **soft BCs** for complex geometries or mixed boundary types.